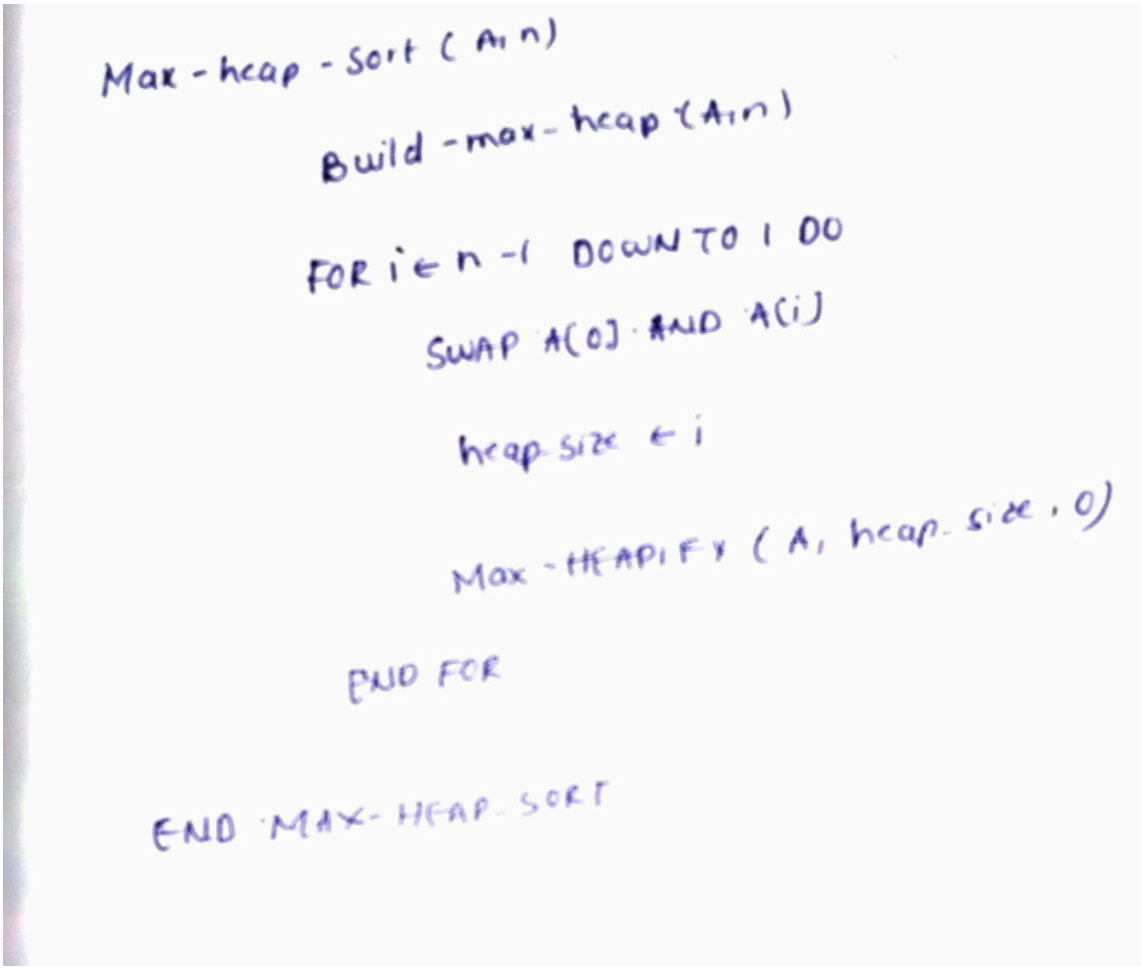
Practical-3	*Implementation of max-heap sort algorithm*
Date :	

Algorithm:



code:

```
#include <iostream>
using namespace std;

// Heapify a subtree rooted at index i
void heapify(int arr[], int n, int i) {
    int largest = i;         // Assume root is largest
    int left = 2 * i + 1;    // Left child
    int right = 2 * i + 2;   // Right child

    // If left child is larger than root
    if (left < n && arr[left] > arr[largest])
        largest = left;

    // If right child is larger than current largest
    if (right < n && arr[right] > arr[largest])
        largest = right;

    // If largest is not the root
    if (largest != i) {
        swap(arr[i], arr[largest]);

        // Recursively heapify the affected subtree
        heapify(arr, n, largest);
    }
}

// Heap Sort
void heapSort(int arr[], int n) {

    // Build Max Heap
    for (int i = n / 2 - 1; i >= 0; i--)
        heapify(arr, n, i);

    // Extract elements one by one
    for (int i = n - 1; i > 0; i--) {

        // Move current largest to the end
        swap(arr[0], arr[i]);

        // Heapify the reduced heap
        heapify(arr, i, 0);
    }
}
```

```
int main() {

    int arr[] = {12, 11, 13, 5, 6, 7};
    int n = sizeof(arr) / sizeof(arr[0]);

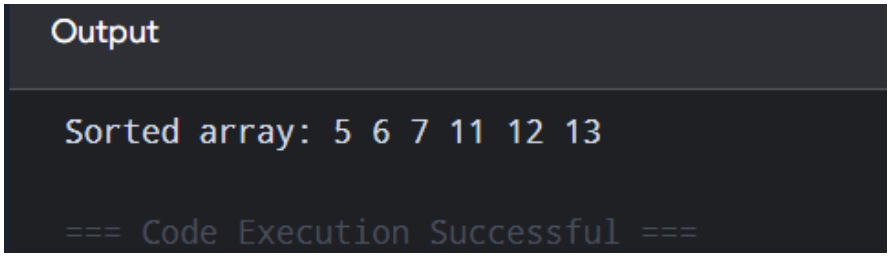
    heapSort(arr, n);

    cout << "Sorted array: ";

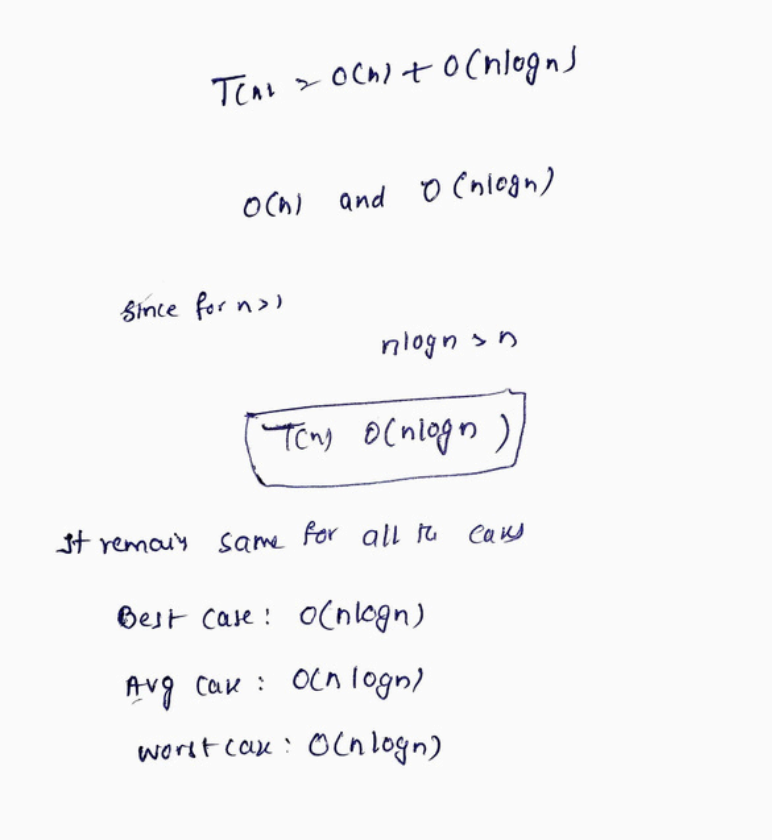
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";

    return 0;
}
```

output:



Time complexity:



Conclusion:

The Max-Heap Sort algorithm was successfully implemented in C++. The practical demonstrated how a Max Heap can be constructed using the heapify() operation and how the maximum element can be repeatedly extracted to sort the array. Heap Sort provides an efficient sorting technique with a time complexity of O(n log n) and requires O(1) auxiliary space when implemented iteratively.